

Desafío 1 — Sweet Crush

Informe — Análisis, diseño e implementación

1. Objetivo del proyecto

El desafío "Sweet Crush" tiene como objetivo gestionar un tablero bidimensional donde el usuario realiza alineaciones de 3 o más fichas iguales, ya sea en horizontal o en vertical, para eliminarlas. Tras cada eliminación se activa un sistema de gravedad que hace caer las fichas superiores para llenar los espacios vacíos, y luego se procesa si se generan combinaciones automáticas en cascada.

2. Restricciones técnicas del proyecto

El desarrollo está sujeto a restricciones estrictas de lenguaje y diseño:

- Prohibido el uso de programación orientada a objetos: struct, class y template.
- Prohibido el uso de arreglos dinámicos de alto nivel, como std::vector, y de cualquier estructura de la biblioteca estándar STL.
- Prohibido el uso de cadenas de texto (string).
- Toda la información del tablero se almacena a nivel de bits: 3 bits por cada ficha, sobre un único bloque continuo de memoria dinámica.

Estas restricciones obligan a que la modularidad del programa se logre agrupando funciones relacionadas en distintos archivos .h/.cpp, en lugar de mediante clases, y a que el estado del juego (el tablero, sus dimensiones, los contadores) se maneje mediante variables sueltas pasadas por puntero o por referencia entre funciones.

3. Análisis del problema: representación en bits

3.1 Mapeo de posición lógica a bits

Se requiere un mapeo matemático exacto para representar la cuadrícula bidimensional de F filas y C columnas en un arreglo unidimensional continuo de bytes. Como la memoria se direcciona en bloques de 8 bits y cada ficha ocupa solo 3 bits, la posición lógica de una ficha no coincide, en general, con los límites físicos de cada byte. Para localizar una ficha a partir de su coordenada (fila, columna) se aplica:

$$\begin{aligned} idx &= (fila \times C) + columna \\ bit_inicio &= idx \times 3 \\ byte_idx &= piso(bit_inicio / 8) \\ bit_offset &= bit_inicio \bmod 8 \end{aligned}$$

Por esta estructura, una ficha puede quedar almacenada de tres maneras distintas: contenida totalmente dentro de un único byte, o dividida entre dos bytes continuos, en proporción 2+1 bits o 1+2 bits, según el valor de bit_offset.

3.2 Tipo de dato para el tablero

Para representar el tablero en memoria dinámica se seleccionó el tipo de dato unsigned char*, asignado dinámicamente en el heap mediante new[]. Un unsigned char ocupa exactamente 8 bits sin signo, por lo que constituye el tipo de dato primitivo más adecuado para actuar como contenedor físico de los bits del tablero.

3.3 Cálculo de bytes mínimos necesarios

La cantidad total de bytes requeridos para almacenar un tablero de FxC se calcula como:

$$Bytes_mínimos = techo((F \times C \times 3) / 8)$$

4. Gestión dinámica de memoria

Durante la ejecución del juego, el tablero puede modificar su estructura mediante la inserción o eliminación de filas y columnas en cualquier posición intermedia, no solo en los extremos. Para equilibrar el rendimiento del programa con el uso responsable de la memoria, se aplican dos políticas: expansión estructural exacta, y reducción sujeta a una regla del 65% de ocupación.

4.1 Expansión estructural

Al insertar una fila o columna, se calcula el nuevo requerimiento exacto de bytes, se asigna un nuevo bloque de memoria con `new[]`, se trasladan los datos preservando el estado de la grilla, y se libera la memoria previa con `delete[]`. La reserva siempre es exacta para las nuevas dimensiones, sin bits sobrantes.

4.2 Reducción con regla del 65%

Al eliminar filas o columnas, no se reasigna memoria física de inmediato, para evitar sobrecostos de procesamiento por fragmentación. La reasignación física y liberación solo se ejecuta si el tamaño de memoria estrictamente necesario cae por debajo del 65% del tamaño reservado en la última asignación física:

$$\text{Porcentaje de uso} = (\text{Bytes_requeridos} / \text{Bytes_reservados}) \times 100\% < 65\%$$

Mientras el porcentaje de uso se mantenga igual o por encima de ese umbral, el bloque de memoria reservado se conserva sin cambios, aunque contenga bits que ya no correspondan a ninguna ficha activa. Esta comparación se implementa con multiplicación cruzada en aritmética entera, evitando el uso de números de punto flotante y cualquier imprecisión de redondeo asociada.

5. Esquema de tareas (arquitectura modular)

Se definieron 8 módulos funcionales (8 pares `.h/.cpp`) más un archivo `main.cpp`, para un total de 17 archivos, organizados en capas: una capa base (bits y memoria) sin dependencias internas, una capa intermedia (tablero, combinaciones, cascadas, estructura) que usa esa base, y una capa superior (juego) que coordina todo.

```
main.cpp -> juego.h/.cpp (orquesta el flujo del juego)
juego --> tablero.h/.cpp (crear, mostrar, eliminar ficha manual)
juego --> estructura.h/.cpp (agregar/eliminar fila o columna)
juego --> estado.h/.cpp (contadores y puntuacion)

tablero --> bits.h/.cpp (leer/escribir 3 bits)
tablero --> memoria.h/.cpp (calcular bytes, reservar, liberar)
tablero --> combinaciones.h/.cpp (verificar que el tablero inicial no nazca con combinaciones)
estructura --> bits.h/.cpp, memoria.h/.cpp, cascadas.h/.cpp
cascadas --> combinaciones.h/.cpp (deteccion y marcado)
cascadas --> bits.h/.cpp (gravedad y relleno)
combinaciones --> bits.h/.cpp (deteccion y marcado)
```

Módulo	Responsabilidad	Depende de
bits	Traduce posición (fila, columna) a bits físicos; lee y escribe el valor de 3 bits de una ficha, incluyendo el caso en que cruza dos bytes. Único módulo que manipula bits directamente.	Ninguno (módulo base)
memoria	Calcula bytes necesarios para FxC fichas, reserva y libera memoria dinámica, y decide cuándo reducir el bloque físico según la ocupación real.	Ninguno (módulo base)
tablero	Crea el tablero inicial (verificando que no nazca con combinaciones ya formadas), lo muestra en formato de fichas y en formato de bits, y procesa la	bits, memoria, combinaciones

	eliminación manual de una ficha.	
combinaciones	Detecta combinaciones horizontales y verticales, y marca las posiciones involucradas reutilizando el código de bits 111.	bits
cascadas	Convierte las marcas a vacío, aplica gravedad y relleno, y repite el ciclo mientras combinaciones detecte nuevas coincidencias, contando las cascadas producidas.	bits, combinaciones
estructura	Agrega o elimina filas/columnas en cualquier posición, reconstruyendo el tablero completo con las nuevas dimensiones.	bits, memoria, cascadas
estado	Mantiene y muestra los contadores: eliminaciones, fichas eliminadas, combinaciones, cascadas y puntuación.	Ninguno (recibe datos por parámetro)
juego	Contiene el menú de opciones y el bucle principal; coordina las llamadas a los demás módulos.	Todos los anteriores
main	Punto de entrada del programa, sin lógica propia.	juego

Las funciones que modifican las dimensiones del tablero reciben un doble puntero (`unsigned char** tablero`) y punteros a filas/columnas, ya que necesitan reemplazar el bloque de memoria completo —reservar uno nuevo, copiar lo necesario y liberar el viejo— y que ese cambio se refleje en la variable original de `juego.cpp`. Ejemplo de firmas representativas:

```
int obtenerFicha(unsigned char* tablero, int columnas, int fila, int columna);
void asignarFicha(unsigned char* tablero, int columnas, int fila, int columna, int valor);
void agregarFila(unsigned char** tablero, int* filas, int* columnas, int posicion);
```

6. Visualización en consola

El programa muestra siempre dos vistas del tablero, con la misma cuadrícula y numeración de filas y columnas: el tablero de fichas (para el usuario) y el tablero de bits (valor binario de 3 dígitos por posición), útil para verificar la representación interna.

<pre> 0 1 2 +---+---+---+ 0 @ # & +---+---+---+ 1 \$. @ +---+---+---+</pre>	Tablero de fichas Tablero de bits	<pre> 0 1 2 +---+---+---+ 0 000 001 010 +---+---+---+ 1 011 110 000 +---+---+---+</pre>
--	---	---

Ambas vistas se construyen con la misma llamada a `obtenerFicha()`; solo cambia si el valor se imprime como símbolo (@, #, &, \$, 0, = para los 6 tipos de ficha, y punto para posición vacía) o como texto binario.

7. Algoritmos implementados

7.1 Lectura y escritura de una ficha

A partir de `byte_idx` y `bit_offset` (sección 3.1), la ficha cabe completa en un solo byte si `bit_offset ≤ 5`. Si `bit_offset` es 6 o 7, la ficha queda repartida entre `byte_idx` y `byte_idx + 1`.

```
Lectura dentro de un solo byte:
valor = (tablero[byte_idx] >> bit_offset) & 0b111
```

```
Lectura cuando la ficha cruza dos bytes:
bitsEnPrimerByte = 8 - bit_offset
parteBaja = (tablero[byte_idx] >> bit_offset) & mascara(bitsEnPrimerByte)
parteAlta = tablero[byte_idx+1] & mascara(3 - bitsEnPrimerByte)
valor = parteBaja | (parteAlta << bitsEnPrimerByte)
```

La escritura sigue el mismo cálculo de posición. Primero se limpian con AND (máscara invertida) únicamente los 3 bits a reemplazar, preservando los bits de las fichas vecinas; luego se inserta el nuevo valor con OR, desplazado a la posición correcta. Si la ficha cruza dos bytes, esta operación de limpiar e insertar se repite en ambos, cada uno con su porción correspondiente del valor.

7.2 Módulo combinaciones: detección y marcado

```
para cada fila (y de forma analoga para cada columna):
    contador = 1
    para cada posicion siguiente:
        si es igual a la anterior y no es VACIO: contador++
        si no: si contador >= 3, marcar las ultimas 'contador' posiciones con codigo 111
            contador = 1
```

Se reutiliza el código 111 (sin ficha asociada en condición normal) como marca temporal, en lugar de un arreglo auxiliar adicional. Así, una ficha compartida entre una combinación horizontal y una vertical queda marcada una sola vez. Tras ambos recorridos, toda posición con valor 111 se convierte a 110 (vacío) en el caso de una eliminación durante el juego.

7.3 Módulo cascadas: gravedad, relleno y ciclo repetitivo

```
convertir marcas (111) a vacio (110)
para cada columna: compactar fichas no vacias hacia abajo, rellenar huecos con fichas nuevas

cascadas = 0
repetir:
    huboCombinacion = combinaciones.detectarYMarcar()
    si huboCombinacion: convertir marcas a vacio; aplicar gravedad y relleno; cascadas++
mientras huboCombinacion
```

Este módulo agrupa las subtarefas posteriores a la detección: conversión de marcas a vacío, gravedad por columna (incluyendo el caso de una columna que queda completamente vacía), relleno con fichas nuevas, repetición del ciclo mientras las fichas nuevas generen combinaciones adicionales, conteo correcto de cascadas, y actualización del estado en cada iteración. Expone una única función hacia el resto del programa —el ciclo completo—, invocada tanto tras la eliminación manual como tras cualquier modificación estructural.

7.4 Redimensionamiento estructural (agregar/eliminar fila o columna)

```
calcular nuevas dimensiones y bytes necesarios
reservar/reallocar memoria segun la politica de la seccion 4
para cada posicion del tablero NUEVO:
    si existia en el tablero viejo: copiar su valor (obtenerFicha/asignarFicha)
    si es una fila/columna nueva: escribir ficha aleatoria
liberar el bloque viejo si aplica; actualizar filas y columnas
ejecutar el ciclo de cascadas sobre el tablero resultante
```

El tablero se almacena como una secuencia continua de bits recorrida fila por fila, por lo que cada fila ocupa un bloque contiguo. Eliminar una fila equivale a quitar un bloque contiguo y desplazar lo siguiente. Una columna, en cambio, es un elemento suelto dentro de cada fila: eliminarla requiere modificar cada fila por separado, y el índice lineal de cada ficha sobreviviente cambia de forma distinta. Por esta razón se usa el mismo algoritmo general de reconstrucción completa para ambos casos, priorizando simplicidad y consistencia sobre una optimización específica para filas.

7.5 Menú de opciones

```
mostrar menu; leer opcion
ELIMINAR FICHA -> tablero.eliminar + cascadas.ejecutarCicloCascadas + estado.actualizar
AGREGAR/ELIMINAR FILA o COLUMNA -> estructura.(funcion correspondiente)
VER ESTADO -> estado.mostrar
SALIR -> termina el bucle
mostrar tablero (ambos formatos) tras cada accion
```

8. Decisiones de diseño

Decisión	Resolución
Codificación de fichas	6 tipos de ficha en códigos 0-5, representados en consola con los símbolos @, #, &, \$, 0, =; código 6 (110) para posición vacía (símbolo punto); código 7 (111) reservado, reutilizado temporalmente como marca de eliminación.
Marcado de posiciones a eliminar	Se reutiliza el código 111 dentro del propio tablero comprimido, evitando un arreglo auxiliar adicional de tamaño FxC.
Generación aleatoria	Se usa la biblioteca estándar de C++ <random> (motor mt19937 y distribución uniforme) para la generación de fichas.
Política de memoria dinámica	Reserva exacta al crear el tablero y al agregar filas/columnas, sin bits sobrantes. Al eliminar, el bloque físico solo se reduce cuando la ocupación real cae por debajo de 65% de la capacidad reservada. El crecimiento se implementa reservando un bloque nuevo, copiando el contenido necesario y liberando el bloque anterior.
Entrada/salida y memoria	Se usan cin/cout y new/delete[] en todo el proyecto, evitando funciones heredadas de C como printf, scanf, malloc o free.
Visualización en consola	Se muestran siempre ambos formatos —tablero de fichas y tablero de bits— con numeración de filas y columnas.
Criterio de puntuación	Se otorgan 10 puntos por cada ficha eliminada como parte de una combinación de 3 o más fichas iguales (horizontal o vertical), incluyendo las cascadas. Las eliminaciones manuales realizadas directamente por el usuario y las fichas eliminadas por modificaciones estructurales (agregar/eliminar fila o columna) no otorgan puntos.

9. Problemas de desarrollo afrontados

Durante la implementación surgieron varios problemas concretos que obligaron a ajustar el diseño inicial. Se documentan a continuación, junto con la solución adoptada:

9.1 Fichas repartidas entre dos bytes

Al implementar la lectura y escritura de una ficha en el núcleo de bits, se hizo evidente que el caso en que una ficha queda repartida entre dos bytes consecutivos (cuando bit_offset es 6 o 7) requiere un tratamiento distinto al caso simple: no basta con desplazar y aplicar una máscara sobre un solo byte, sino que hay que extraer una parte de cada byte y combinarlas con un desplazamiento adicional. De igual forma, la escritura no puede sobrescribir el byte completo, ya que contiene bits de fichas vecinas que no deben alterarse; fue necesario aplicar primero una máscara invertida para limpiar únicamente los bits correspondientes, y luego insertar el nuevo valor con OR.

9.2 Pérdida de combinaciones compartidas entre direcciones

Al probar el módulo de detección de combinaciones con casos donde una misma ficha pertenece simultáneamente a una combinación horizontal y a una vertical (por ejemplo, formando una cruz o una L), se detectó que la combinación en la segunda dirección de recorrido no se reconocía correctamente. La causa fue que el recorrido horizontal marca las posiciones que encuentra con el código temporal de eliminación antes de que el recorrido vertical las evalúe, y la comparación de valores fallaba al toparse con esa marca en lugar del valor original de la ficha. La solución adoptada fue tratar una posición ya marcada como compatible con la racha en curso en la otra dirección, ya que de todas formas esa posición va a eliminarse.

9.3 Conteo insuficiente en la detección de combinaciones

La primera versión del módulo de combinaciones únicamente indicaba si se había encontrado alguna combinación, sin precisar cuántas. Al integrar el contador de "combinaciones detectadas" exigido en el estado del juego, se identificó que esta información era insuficiente, ya que un mismo recorrido puede encontrar varias combinaciones distintas a la vez (por ejemplo, dos rachas separadas en la misma fila). Se ajustaron las funciones del módulo para que devolvieran la cantidad exacta de combinaciones marcadas en lugar de un valor booleano.

9.4 Relleno pendiente tras una eliminación manual

En el diseño inicial del ciclo de cascadas, la gravedad y el relleno solo se aplicaban después de detectar una combinación. Esto dejaba sin resolver el hueco que deja una eliminación manual simple, ya que retirar una sola ficha no puede por sí sola generar una combinación nueva. Fue necesario incorporar una gravedad y relleno inicial obligatorios antes de entrar al ciclo de detección, de modo que cualquier hueco existente se llene independientemente de si se detecta o no una combinación posterior.

9.5 Seguimiento de la capacidad física reservada

Al implementar la modificación estructural del tablero (agregar o eliminar filas y columnas), se identificó que comparar el nuevo tamaño requerido contra un recálculo hecho a partir de las dimensiones lógicas actuales no reflejaba correctamente la memoria físicamente reservada, en los casos donde una eliminación anterior había decidido no reducir el bloque (por no cumplirse la regla del 65%). Esto podía hacer que la regla dejara de aplicarse correctamente a lo largo de varias eliminaciones sucesivas. La solución fue introducir una variable independiente que registra la capacidad física realmente reservada en cada momento, actualizada explícitamente por cada operación de crecimiento (siempre exacta) o reducción (solo cuando corresponde según la regla).

9.6 Consistencia entre la documentación y la implementación

Durante la integración de los módulos de visualización se detectó una discrepancia entre los símbolos usados en consola para representar las fichas y los definidos en el informe preliminar. Se corrigió el código para que coincidiera exactamente con lo documentado, evitando una inconsistencia entre lo entregado como análisis y lo efectivamente implementado.

9.7 Tablero inicial con combinaciones ya formadas

Al probar el juego de principio a fin, se detectó que el llenado aleatorio del tablero inicial podía, por azar, generar combinaciones de 3 o más fichas ya alineadas antes de que el usuario realizara cualquier movimiento — un comportamiento inconsistente con la mecánica de un juego tipo match-3, donde el tablero de partida debe estar estable. La solución fue que, tras el llenado aleatorio inicial, `tablero.cpp` invoque al módulo combinaciones para detectar si existen coincidencias; de haberlas, únicamente las posiciones marcadas se regeneran con un nuevo valor aleatorio en el mismo lugar (sin aplicar gravedad, ya que no se trata de una eliminación del jugador), repitiendo la verificación hasta que el tablero quede sin combinaciones. Esto introdujo una dependencia nueva del módulo tablero hacia combinaciones, no contemplada en el diseño original.

10. Evolución de la solución

El desarrollo se llevó a cabo de forma incremental, construyendo primero los módulos base y verificando cada uno de forma aislada antes de construir funcionalidad adicional sobre ellos:

- Núcleo de bits (lectura y escritura de una ficha): primer módulo implementado y probado de forma independiente, ya que todos los demás dependen de él.
- Gestión de memoria (cálculo de bytes, reserva, liberación, regla del 65%): construido sobre la base anterior, sin depender de ningún otro módulo.
- Tablero (creación, visualización en ambos formatos, eliminación manual): primera funcionalidad visible del juego; revisado posteriormente para verificar que el tablero inicial no naciera con combinaciones, según lo descrito en la sección 9.7.
- Detección de combinaciones: revisado durante la integración para pasar de un resultado booleano a un conteo, según lo explicado en la sección 9.3.
- Cascadas (gravedad, relleno, ciclo completo): incorporó la corrección de la gravedad inicial obligatoria descrita en la sección 9.4.
- Estado del juego: motivó el ajuste al módulo de combinaciones al evidenciar la necesidad de un conteo exacto.
- Modificación estructural: evidenció la necesidad del seguimiento explícito de la capacidad reservada, descrito en la sección 9.5.
- Integración final (menú y punto de entrada): unió todos los módulos anteriores en un programa funcional completo, validado con una serie de pruebas manuales y automatizadas que cubren eliminación manual, combinaciones horizontales y verticales, modificaciones estructurales en posiciones intermedias, cascadas sucesivas, y la generación correcta del tablero inicial.

Como consideración general para la implementación, se privilegió una arquitectura en capas, con los módulos de bits y memoria como base sin dependencias entre sí. Esto permitió que los problemas encontrados en etapas posteriores del desarrollo (secciones 9.2 a 9.7) se pudieran aislar y corregir en el módulo correspondiente, sin necesidad de modificar el núcleo de bits ya validado desde el principio.